



**Instituto Tecnológico y de Estudios Superiores de Monterrey**

Campus Monterrey, Escuela de Ingeniería y Ciencias, IDM

*MA2006B — Uso de Álgebras Modernas para Seguridad y Criptografía*

## **Reporte Técnico**

**Instar: Gestor de Identidades, Registros y Documentos**

Socio Formador

**Casa Monarca — Ayuda Humanitaria al Migrante, A.B.P.**

**Reto:** Firmas solidarias y protección de datos personales

**Equipo:** Equipo 4 Grupo: Grupo 602

**Versión del sistema:** v1.0.0

**Profesores:** Alberto F. Martínez, Daniel Otero Fadul, Raúl Gómez Muñoz, Pedro Leonel Olaya Trejos

### **Integrantes del equipo**

Juan David Pastrana Arango, A01751471, Líder de desarrollo

Yamil Elias Del Blanco Chávez, A00838610, Líder técnico

Luis Roberto Campos Solis, A00838686, Líder de equipo

Mariano Vertiz Sánchez, A00840430, Desarrollador

Álvaro Duhart Pérez, A01029468, Desarrollador

Mauro Artemio Sotelo Dávila, A01707689, Desarrollador

**Monterrey, Nuevo León · 14 de junio de 2026**

# Índice de contenidos

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| Índice de contenidos                                                    | 2  |
| Índice de tablas                                                        | 4  |
| Índice de figuras                                                       | 5  |
| Abstract                                                                | 6  |
| 1. Introducción                                                         | 7  |
| 1.1. Estructura de las entregas                                         | 7  |
| 2. Marco de referencia                                                  | 9  |
| 2.1. Criptografía de llave pública y RSA                                | 9  |
| 2.2. Funciones hash y SHA-256                                           | 9  |
| 2.3. HMAC-SHA256 para sellado de integridad                             | 10 |
| 2.4. Protocolo challenge-response y Web Crypto API                      | 10 |
| 2.5. Infraestructura de Llave Pública (PKI)                             | 10 |
| 2.6. Marco normativo mexicano                                           | 11 |
| 2.7. Soluciones existentes revisadas                                    | 11 |
| 3. Desarrollo                                                           | 13 |
| 3.1. Propuesta de solución                                              | 13 |
| 3.1.1. Cobertura del reto                                               | 14 |
| 3.2. Desarrollo de componentes (v1.0.0)                                 | 14 |
| 3.2.1. Backend                                                          | 14 |
| 3.2.2. Frontend                                                         | 17 |
| 3.2.3. Conexión de los componentes                                      | 18 |
| 3.3. Características adicionales en v1.1.0 (Etapa 2)                    | 19 |
| 3.3.1. Firma digital challenge-response de documentos (FirmaController) | 19 |
| 3.3.2. Sellado de integridad de documentos de identidad                 | 20 |
| 3.3.3. Flujo completo de Corrección ARCO                                | 21 |
| 3.3.4. Visibilidad en dos fases para documentos de expediente           | 22 |
| 3.3.5. Nuevo rol: Voluntario (nivel 4)                                  | 23 |
| 3.3.6. Vista de auditoría de archivos (/admin/archivos)                 | 23 |
| 3.3.7. Creación directa de casos por Coordinador                        | 23 |
| 3.3.8. Cambios de infraestructura                                       | 24 |
| 3.3.9. Stack tecnológico — cambios entre v1.0.0 y v1.1.0                | 24 |
| 3.3.10. Endpoints nuevos en v1.1.0                                      | 25 |
| 4. Resultados                                                           | 27 |
| 4.1. Resultados de backend                                              | 27 |
| 4.2. Resultados de frontend                                             | 28 |
| 4.3. Resultados de la solución completa                                 | 30 |
| 4.4. Casos de uso y matriz de validación                                | 31 |
| 4.5. Cobertura criptográfica por servicio de seguridad                  | 33 |
| 5. Conclusiones y trabajo futuro                                        | 35 |
| 5.1. Conclusiones                                                       | 35 |

|                                                                  |    |
|------------------------------------------------------------------|----|
| 5.2. Trabajo futuro                                              | 35 |
| Referencias                                                      | 37 |
| Anexos                                                           | 39 |
| Anexo A. Casos de uso principales                                | 39 |
| Anexo B. Pseudocódigo de los flujos criptográficos clave         | 40 |
| B.1. Emisión de certificado RSA-2048 al aprobar a un Coordinador | 40 |
| B.2. Verificación challenge-response (v1.1.0)                    | 40 |
| B.3. Sellado HMAC-SHA256 de documento de identidad (v1.1.0)      | 41 |
| Anexo C. Stack tecnológico completo v1.1.0                       | 42 |
| Anexo D. Glosario técnico                                        | 43 |
| Anexo E. Link de Github                                          | 43 |

## Índice de tablas

|                                                                                 |        |
|---------------------------------------------------------------------------------|--------|
| <b>Tabla 1.</b> Cobertura del reto por sub-problema en v1.1.0.                  | pág 13 |
| <b>Tabla 2.</b> Estados del flujo de Corrección ARCO.                           | pág 21 |
| <b>Tabla 3.</b> Niveles de acceso y permisos del sistema v1.1.0.                | pág 22 |
| <b>Tabla 4.</b> Stack tecnológico — cambios entre v1.0.0 y v1.1.0.              | pág 23 |
| <b>Tabla 5.</b> Endpoints REST nuevos en v1.1.0.                                | pág 24 |
| <b>Tabla 6.</b> Matriz funcional de validación de casos de uso (TC-01 a TC-22). | pág 30 |
| <b>Tabla 7.</b> Cobertura criptográfica por servicio de seguridad.              | pág 32 |
| <b>Tabla 8.</b> Casos de uso principales del sistema.                           | pág 36 |

# Índice de figuras

|                                                                               |        |
|-------------------------------------------------------------------------------|--------|
| <b>Fig. 1.</b> Jerarquía de niveles de acceso y permisos CRUD.                | pág 11 |
| <b>Fig. 2.</b> Pantalla de bienvenida con selección del tipo de cuenta.       | pág 12 |
| <b>Fig. 3.</b> Arquitectura MVC del sistema en Laravel.                       | pág 14 |
| <b>Fig. 4.</b> Flujo de emisión de certificado RSA-2048.                      | pág 15 |
| <b>Fig. 5.</b> Flujo de verificación PEM en acciones sensibles (v1.0.0).      | pág 16 |
| <b>Fig. 6.</b> Bandeja de Accesos del Administrador.                          | pág 17 |
| <b>Fig. 7.</b> Pantalla de entrega única de llave privada PEM.                | pág 17 |
| <b>Fig. 8.</b> Diagrama conceptual del protocolo challenge-response RSA-2048. | pág 19 |
| <b>Fig. 9.</b> Flujo de sellado HMAC-SHA256 de documentos de identidad.       | pág 20 |
| <b>Fig. 10.</b> Máquina de estados del flujo de Corrección ARCO.              | pág 21 |
| <b>Fig. 11.</b> Bloqueo de acceso por estado distinto de alta.                | pág 26 |
| <b>Fig. 12.</b> Ciclo de vida de una identidad digital.                       | pág 27 |
| <b>Fig. 13.</b> Panel del Coordinador.                                        | pág 28 |
| <b>Fig. 14.</b> Formulario de registro de colaborador interno.                | pág 28 |
| <b>Fig. 15.</b> Entrevista de ingreso al albergue para personas migrantes.    | pág 29 |
| <b>Fig. 16.</b> Modelo entidad-relación del sistema.                          | pág 30 |

## Abstract

El presente reporte documenta el sistema Instar: Gestor de Identidades, Registros y Documentos, desarrollado para Casa Monarca como Socio Formador del bloque MA2006B del Tecnológico de Monterrey. El documento consolida los resultados de las dos primeras etapas del reto "Firmas solidarias y protección de datos personales": la Etapa 1 (versión 1.0.0) que estableció el ciclo de vida completo de identidades digitales y una infraestructura interna de llave pública basada en RSA-2048, y la Etapa 2 (versión 1.1.0) que cierra brechas de la verificación criptográfica mediante un protocolo challenge-response de firma digital, incorpora sellado de integridad HMAC-SHA256 sobre documentos de identidad, y entrega el flujo operativo completo del Derecho ARCO de Rectificación.

La arquitectura del sistema permanece como un monolito Laravel 13 sobre PHP 8.3 con plantillas Blade, Tailwind CSS y Alpine.js. El motor criptográfico server-side continúa con OpenSSL nativo de PHP, y en la Etapa 2 se introduce firma RSA-2048 en el navegador mediante la Web Crypto API para que la llave privada del Coordinador nunca viaje al servidor, ni siquiera durante el proceso de firma. Adicionalmente, la versión 1.1.0 introduce un sexto rol (Voluntario, nivel 4) con permisos exclusivos de creación, migra la base de datos de desarrollo a SQLite para simplificar el onboarding del equipo, y entrega una vista administrativa de auditoría de archivos por migrante con verificación de integridad en tiempo real desde disco.

El sistema mantiene una bitácora inmutable de auditoría que preserva snapshot textual del actor incluso ante eliminación posterior del usuario, alineada con el Derecho ARCO de Cancelación. Los resultados confirman la viabilidad técnica de digitalizar los procesos administrativos de la organización con un nivel de garantía criptográfica equivalente a la e.firma del SAT en lo referente al no repudio de las acciones del personal coordinador, sin comprometer la confidencialidad de los datos personales bajo su custodia. Se validaron veintidós casos de uso end-to-end (TC-01 a TC-22), de los cuales siete corresponden a las nuevas capacidades de la Etapa 2.

# 1. Introducción

Casa Monarca, Ayuda Humanitaria al Migrante A.B.P., es una organización de la sociedad civil sin fines de lucro que brinda acompañamiento humanitario, psicosocial, legal y de comunicación a personas migrantes y refugiadas en su tránsito por el norte de México. En su operación cotidiana, la organización maneja datos personales sensibles —situación migratoria, estado de salud, historia familiar, documentos de identidad y constancias oficiales— que la Ley Federal de Protección de Datos Personales en Posesión de los Particulares y los Derechos ARCO obligan a custodiar con estándares específicos de integridad, confidencialidad, trazabilidad y disponibilidad (Cámara de Diputados, 2025).

Hasta antes del desarrollo de este sistema, la mayor parte de los procesos de altas, asignaciones, autorizaciones y bajas del personal colaborador se gestionaban mediante hojas de cálculo y correos electrónicos, sin un control centralizado de quién tenía acceso a qué información, sin una manera robusta de demostrar la autenticidad de los documentos emitidos, y sin un registro inalterable de las acciones realizadas. Este vacío representa un riesgo legal y operativo serio: impide responder de manera ágil a una solicitud de Derechos ARCO de un titular y dificulta demostrar ante una eventual auditoría que la organización efectivamente protege los datos que custodia.

El reto "Firmas solidarias y protección de datos personales" contempla siete sub-problemas posibles que cubren distintas dimensiones de la seguridad de la información en organizaciones sin fines de lucro. Este equipo aborda principalmente el Sub-problema 1 (Gestor de Identidades) y el Sub-problema 5 (gestión de expedientes y solicitudes), entregando un sistema que: (a) administra el ciclo de vida completo de las identidades del personal; (b) emite certificados digitales RSA-2048 al momento de aprobar a un Coordinador; (c) garantiza que la llave privada de cada Coordinador nunca quede almacenada en el servidor; (d) verifica criptográficamente, mediante firma challenge-response, que la persona que ejecuta una acción sensible posee efectivamente la llave privada correspondiente; (e) mantiene una bitácora inmutable de toda actividad relevante para auditoría y cumplimiento ARCO; y (f) provee un flujo completo de Rectificación ARCO con aprobación criptográfica del Coordinador.

## 1.1. Estructura de las entregas

El sistema fue concebido como un desarrollo multi-semestral con releases incrementales. Este reporte consolida las dos primeras versiones liberadas:

- **Versión 1.0.0 (Etapa 1, cierre 30 abril 2026).** Estableció el ciclo de vida de identidades, la emisión de certificados RSA-2048, la verificación de posesión de llave PEM mediante comparación de fingerprints SHA-256, y la bitácora inmutable.
- **Versión 1.1.0 (Etapa 2, cierre 9 junio 2026).** Elevó la verificación criptográfica a una firma digital genuina sobre el contenido mediante un protocolo challenge-response basado en Web Crypto API, añadió sellado HMAC-SHA256 sobre documentos de identidad, entregó el flujo operativo

completo de Rectificación ARCO, incorporó el rol de Voluntario y simplificó la infraestructura de desarrollo.

El presente documento describe el problema abordado, el marco de referencia técnico y normativo, la propuesta de solución, los componentes desarrollados en backend y frontend de ambas etapas, los resultados obtenidos, los casos de uso validados y las direcciones de trabajo futuro hacia las etapas posteriores del reto.

## **2. Marco de referencia**

### **2.1. Criptografía de llave pública y RSA**

La criptografía de llave pública es una rama de la criptografía que utiliza pares de llaves matemáticamente relacionadas, donde una llave se difunde abiertamente (llave pública) y otra se mantiene en secreto absoluto por su titular (llave privada). Cualquier mensaje cifrado con la llave pública sólo puede ser descifrado con la llave privada correspondiente, y viceversa, lo cual permite construir esquemas de confidencialidad y de firma digital sin necesidad de compartir un secreto previo entre las partes (Menezes et al., 1996).

RSA, propuesto por Rivest, Shamir y Adleman (1978), es uno de los algoritmos asimétricos más utilizados a nivel mundial. Su seguridad descansa en la dificultad computacional del problema de la factorización de enteros grandes. De acuerdo con Barker (2020), un módulo RSA de 2048 bits ofrece una fortaleza equivalente a 112 bits de seguridad simétrica, considerado aceptable para la mayoría de aplicaciones de propósito general hasta más allá del año 2030. El sistema utiliza RSA-2048 como tamaño de llave para los certificados internos del personal coordinador, tanto en operaciones server-side (emisión y verificación de posesión, v1.0.0) como en operaciones client-side (firma challenge-response, v1.1.0).

### **2.2. Funciones hash y SHA-256**

Una función hash criptográfica toma una entrada de longitud arbitraria y produce una salida de longitud fija (resumen), con la propiedad de que es computacionalmente inviable invertir el cálculo, encontrar dos entradas que produzcan la misma salida, o predecir cómo cambia la salida ante un cambio mínimo en la entrada. SHA-256 (Secure Hash Algorithm, variante de 256 bits) es la función hash predominante para integridad y huellas digitales en aplicaciones modernas (Paar y Pelzl, 2010). En este sistema, SHA-256 se utiliza con cinco propósitos:

1. Generar el fingerprint identificador único de cada llave pública RSA emitida (v1.0.0).
2. Verificar criptográficamente que un archivo PEM ingresado por el usuario corresponde efectivamente al certificado activo en base de datos antes de autorizar una acción sensible (v1.0.0).

3. Calcular y persistir la huella de integridad de cada documento adjunto cargado en un expediente (v1.0.0).
4. Servir como función subyacente del HMAC para sellado de documentos de identidad (v1.1.0).
5. Componer el digest del challenge en el protocolo de firma RSASSA-PKCS1-v1\_5-SHA256 (v1.1.0).

### **2.3. HMAC-SHA256 para sellado de integridad**

HMAC (Hash-based Message Authentication Code), descrito en el RFC 2104 (Krawczyk et al., 1997), extiende una función hash criptográfica incorporando una llave secreta en el cálculo del resumen. El resultado no puede ser replicado sin conocer la llave, lo que permite verificar simultáneamente la integridad del dato original y la autenticidad del proceso que lo generó. A diferencia de un hash simple, HMAC es resistente a ataques de extensión de longitud y a colisiones inducidas, lo que lo hace adecuado para sellos de autenticidad sobre datos almacenados.

En la versión 1.1.0, HMAC-SHA256 se utiliza para sellar la integridad de los documentos de identidad subidos por las personas migrantes. La llave secreta es la APP\_KEY de Laravel, una cadena aleatoria de 256 bits generada al inicializar el ambiente. Esta llave es conocida sólo por el servidor y le permite tanto generar como verificar sellos válidos. La existencia del sello no garantiza solamente que el archivo no haya sido modificado en disco, sino también que el sello fue emitido por la aplicación legítima — no por un atacante que hubiera obtenido únicamente el hash del archivo.

### **2.4. Protocolo challenge-response y Web Crypto API**

Un protocolo challenge-response es un mecanismo de autenticación criptográfica en el cual una parte (el verificador) envía un dato fresco y aleatorio (el challenge, también llamado nonce) a la parte que desea probar su identidad (el prover). Esta última firma o transforma criptográficamente el challenge con un secreto que sólo ella conoce, y devuelve el resultado. El verificador comprueba la respuesta usando información pública del prover. La propiedad clave del esquema es que cada autenticación utiliza un challenge distinto, por lo que no es susceptible de ataques de replay donde un atacante hubiera capturado una respuesta anterior.

La Web Crypto API es una interfaz nativa de los navegadores modernos definida por el W3C que expone primitivas criptográficas estándar al código JavaScript. Esta API permite generar pares de llaves, importar llaves desde formatos estándar como PKCS#8, firmar y verificar mensajes, y derivar llaves, sin necesidad de bibliotecas externas y con implementaciones acreditadas a nivel del navegador (Mozilla, 2025). En la versión 1.1.0, la Web Crypto API se utiliza para implementar la firma RSASSA-PKCS1-v1\_5 con digest SHA-256 sobre el challenge generado por el servidor, garantizando que la llave privada del Coordinador nunca abandone el navegador del cliente.

## 2.5. Infraestructura de Llave Pública (PKI)

Una PKI es el conjunto de roles, políticas, hardware, software y procedimientos necesarios para crear, gestionar, distribuir, usar, almacenar y revocar certificados digitales. El estándar de facto para certificados es X.509, descrito en el RFC 5280 (Cooper et al., 2008), el cual también define el formato de las Listas de Revocación de Certificados (CRL). En una PKI completa, una Autoridad Certificadora (CA) firma los certificados de los usuarios finales, validando que la llave pública incluida pertenece efectivamente al titular declarado.

En las Etapas 1 y 2 de este proyecto se implementa una PKI interna simplificada en la cual el propio sistema actúa como autoridad emisora para los certificados del personal coordinador. No se integra todavía con una Autoridad Certificadora externa ni se exportan CRL en el formato estándar RFC 5280. Esta decisión se justifica por el alcance multi-semestral del reto: las etapas posteriores incorporarán formatos estándar y eventualmente una capa de registro inmutable basada en blockchain para los eventos criptográficos relevantes.

## 2.6. Marco normativo mexicano

La Ley Federal de Protección de Datos Personales en Posesión de los Particulares (Cámara de Diputados, 2025) regula el tratamiento de datos personales por parte de organizaciones privadas en México. En su artículo 6 enuncia los principios rectores de licitud, consentimiento, información, calidad, finalidad, lealtad, proporcionalidad y responsabilidad. En los artículos 22 a 27 detalla los Derechos ARCO: el titular de los datos tiene derecho a acceder a la información que la organización mantiene sobre él, a rectificarla cuando sea inexacta o incompleta, a cancelar el tratamiento cuando ya no resulte necesario, y a oponerse al uso de sus datos para fines específicos por causa legítima (INAI, 2023).

El diseño del sistema considera estos derechos de manera operativa. La versión 1.0.0 incorporó las bases técnicas para responder a las solicitudes de Acceso (trazabilidad inmutable) y Cancelación (estados de baja y revocación con bitácora persistente). La versión 1.1.0 completa la cobertura del derecho de Rectificación mediante un flujo dedicado de SolicitudRectificacion que permite al migrante iniciar la solicitud, al Operativo recibir el documento corregido, y al Coordinador aprobar con firma criptográfica, manteniendo registro inmutable de ambas versiones (la eliminada y la nueva) en actividad\_log. El procedimiento formal de Oposición a usos específicos queda señalado como pendiente para etapas posteriores.

## 2.7. Soluciones existentes revisadas

Durante el desarrollo se revisaron varias soluciones de referencia. El sistema e.firma del Servicio de Administración Tributaria (SAT) opera como un caso de PKI ciudadana en México: el contribuyente solicita un par de llaves, el SAT actúa como CA, y la llave privada es responsabilidad del titular. Este modelo inspira el patrón de entrega única de PEM utilizado en este sistema. La biblioteca PyCryptodome

de Python fue evaluada como alternativa para la capa criptográfica; sin embargo, dado que el stack final del proyecto fue PHP con Laravel, se optó por OpenSSL nativo de PHP (The PHP Group, 2025), que ofrece equivalencia funcional. Adicionalmente, la herramienta PGP (Pretty Good Privacy) sirvió como referencia conceptual para la separación entre operaciones de firma y de cifrado.

Para la firma client-side de la Etapa 2 se revisaron implementaciones de referencia que utilizan la Web Crypto API, en particular las recomendaciones de Mozilla Developer Network para RSASSA-PKCS1-v1\_5 (Mozilla, 2025). También se estudiaron sistemas con control de acceso por roles y privilegios mínimos como referente operativo de cómo segregar permisos entre administradores, coordinadores, operativos, voluntarios y migrantes. La Fig. 1 muestra la jerarquía resultante implementada en el sistema.

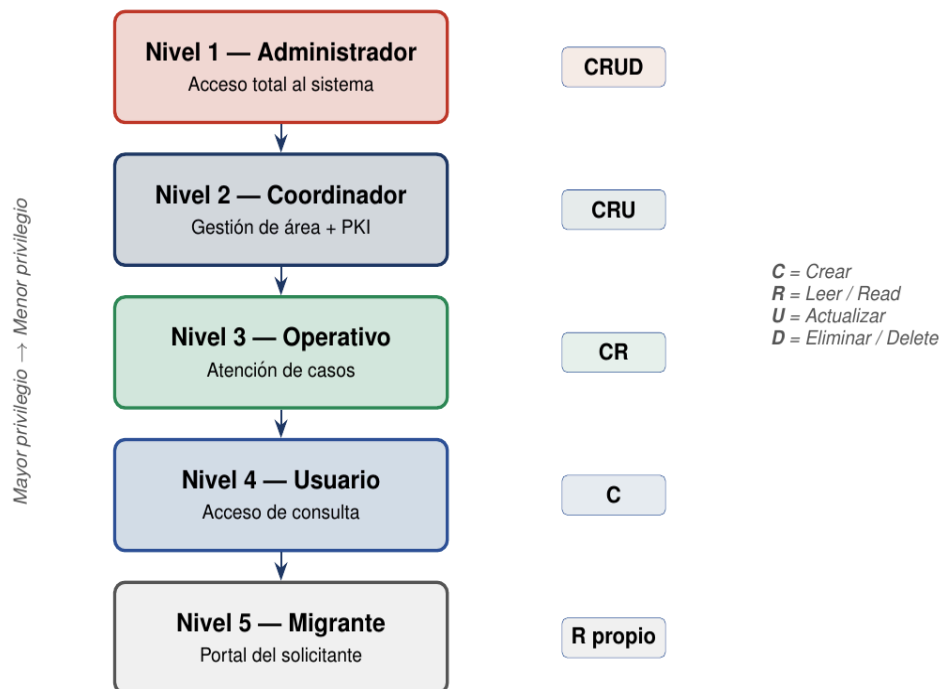


Fig. 1. Jerarquía de niveles de acceso y permisos CRUD (elaboración propia).

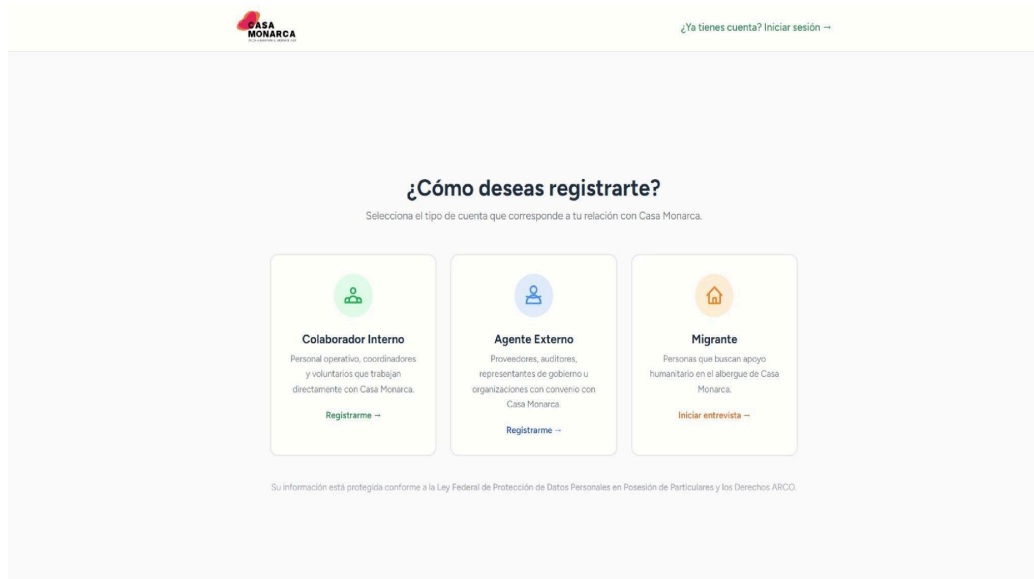
## 3. Desarrollo

### 3.1. Propuesta de solución

La propuesta consiste en una aplicación web monolítica construida sobre Laravel 13, accesible desde navegador, con autenticación por correo y contraseña como factor primario. La aplicación gestiona cinco niveles de acceso para personal interno —Administrador, Coordinador, Operativo, Voluntario y Usuario— más un nivel adicional para personas migrantes con su portal específico, y distribuye estos roles sobre seis áreas operativas de Casa Monarca: Humanitaria, Psicosocial, Legal, Comunicación, Almacén y

Tecnologías de Información. Cada acción relevante queda registrada en una bitácora inmutable que conserva una copia textual del actor incluso si el usuario es eliminado posteriormente del sistema.

La Fig. 2 muestra la pantalla de bienvenida del sistema, donde el visitante elige el tipo de cuenta con la que se relacionará con Casa Monarca. La distinción visual entre Colaborador Interno, Agente Externo y Migrante refuerza desde el primer contacto el modelo de roles del sistema.



*Fig. 2. Pantalla de bienvenida con selección del tipo de cuenta (elaboración propia).*

La elección de Laravel se justifica por varias razones técnicas concretas. Primero, el bloque MA2006B no se centra en la construcción de un framework propio sino en la aplicación correcta de algoritmos criptográficos modernos; Laravel resuelve de fábrica la autenticación, el ruteo, el control de sesiones, las políticas de autorización (Gates y Policies), la protección contra falsificación de peticiones (CSRF), el ORM relacional (Eloquent) y las migraciones de base de datos versionadas (Laravel, 2025). Segundo, la extensión OpenSSL viene incluida nativamente en PHP, lo cual evita instalar dependencias externas para la generación y validación de pares RSA. Tercero, la curva de aprendizaje del equipo era favorable, permitiendo concentrar el esfuerzo en la capa criptográfica y los flujos de negocio. Cuarto, las plantillas Blade junto con Tailwind CSS (Tailwind Labs, 2025) y Alpine.js (Alpine.js, 2025) permiten construir interfaces consistentes sin requerir una aplicación frontend separada, y la integración con la Web Crypto API se logra naturalmente desde Alpine.

Para el hosting se evaluó la posibilidad de despliegue en proveedores tipo HostGator o servicios cloud completos como AWS. Dado que el sistema se entrega en estado piloto sobre desarrollo local, la decisión final de proveedor de producción se documenta como abierta. Se descartó AWS para esta etapa por la complejidad innecesaria que introduce su modelo de servicios diferenciados frente al alcance acotado del piloto; en cambio, una solución tipo HostGator o un VPS modesto cubre los requerimientos actuales con menor costo y menor complejidad operativa para Casa Monarca. La base de datos de producción se mantiene en MySQL 8 con orquestación Docker.

### 3.1.1. Cobertura del reto

La tabla 1 resume qué sub-problemas del reto están cubiertos al cierre de la versión 1.1.0:

| SP   | Descripción                                                       | Cobertura v1.1.0                |
|------|-------------------------------------------------------------------|---------------------------------|
| SP-1 | Gestor de Identidades: ciclo de vida con certificados y auditoría | Principal (extendido en v1.1.0) |
| SP-2 | Correos cifrados                                                  | Fuera de alcance                |
| SP-3 | Registros inmutables externos                                     | Fuera de alcance                |
| SP-4 | Verificación externa de documentos                                | Fuera de alcance                |
| SP-5 | Gestión de expedientes y solicitudes                              | Extendido — firma digital real  |
| SP-6 | Cifrado de base de datos                                          | Fuera de alcance                |
| SP-7 | Respaldos automáticos                                             | Fuera de alcance                |

*Tabla 1. Cobertura del reto por sub-problema en v1.1.0 (elaboración propia).*

## 3.2. Desarrollo de componentes (v1.0.0)

### 3.2.1. Backend

El backend se organiza en módulos funcionales claramente delimitados. Cada módulo agrupa controladores, modelos Eloquent, políticas de autorización y servicios específicos. La Fig. 3 ilustra la arquitectura MVC adoptada. La siguiente figura (figura 3) muestra la Arquitectura MVC del sistema en Laravel, con la organización mencionada anteriormente.

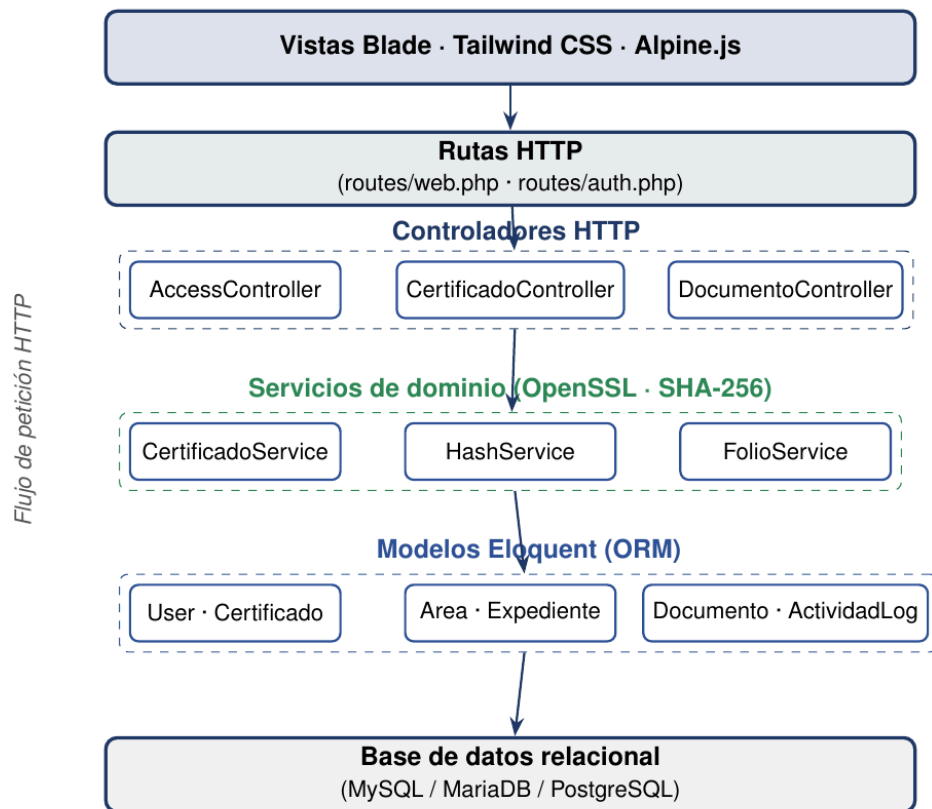


Fig. 3. Arquitectura MVC del sistema en Laravel (elaboración propia).

**Autenticación y sesión.** La autenticación se construye sobre Laravel Breeze. Para el inicio de sesión se utiliza correo institucional o personal del colaborador y contraseña. El middleware CheckStatus se aplica globalmente a las rutas autenticadas y bloquea el acceso de cuentas con estado distinto de "alta", redirigiendo a una pantalla informativa.

**Gestión de usuarios e identidades.** Cada usuario en el sistema tiene un rol numérico, un área asignada cuando aplica, y un estado dentro del ciclo de vida (pendiente, alta, suspendido, revocación, baja). El registro inicial coloca a la persona en estado pendiente y dispara una notificación al Administrador. Desde la Bandeja de Accesos, el Administrador puede aprobar, rechazar, suspender, revocar o restaurar identidades.

**Certificados digitales.** Al aprobar a un Coordinador, el sistema invoca openssl\_pkey\_new() para generar un par RSA-2048. La llave pública se almacena en la tabla certificados junto con su fingerprint SHA-256. La llave privada, en formato PEM, se introduce en una variable flash de sesión y se renderiza una sola vez en una pantalla de confirmación. La Fig. 4 ilustra este flujo paso a paso.

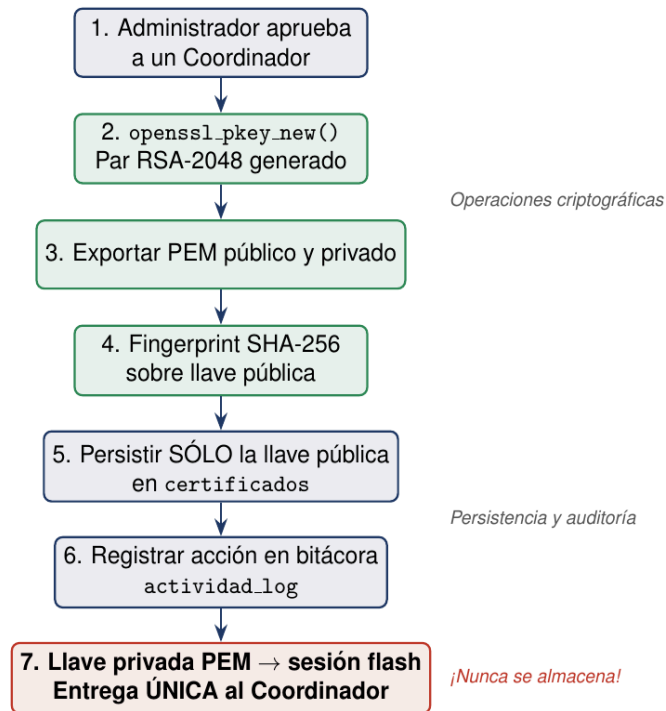
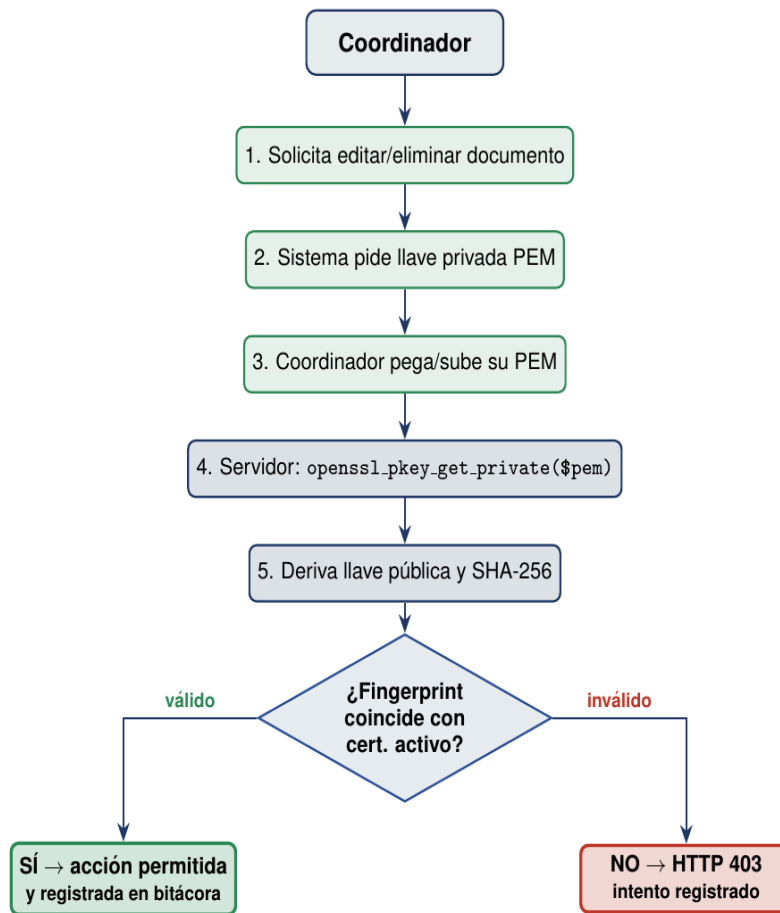


Fig. 4. Flujo de emisión de certificado RSA-2048 (elaboración propia).

**Verificación criptográfica PEM (v1.0.0).** En la primera versión, las acciones sensibles (edición y eliminación de documentos en un expediente) requerían que el usuario introdujera su llave privada PEM. El servidor: (1) cargaba el certificado activo del usuario; (2) extraía la llave pública y calculaba su fingerprint SHA-256; (3) derivaba del PEM introducido la llave pública correspondiente y calculaba su fingerprint; (4) comparaba los dos fingerprints con `hash_equals()` para evitar timing attacks. La Fig. 5 muestra el flujo conceptual. *Esta verificación demostraba posesión de la llave privada pero no era una firma digital sobre el contenido; este punto se elevó en la Etapa 2 con el protocolo challenge-response descrito en la sección 3.3.1.*



*La comparación usa `hash_equals()` para evitar timing attacks.  
El PEM ingresado nunca se persiste en servidor.*

*Fig. 5. Flujo de verificación PEM en acciones sensibles, versión 1.0.0 (elaboración propia).*

**Auditoría inmutable.** La tabla `actividad_log` conserva el rastro de toda acción relevante. A diferencia de las relaciones tradicionales, los registros de bitácora no usan llave foránea hacia el usuario que ejecutó la acción; en su lugar, almacenan una copia textual del nombre, matrícula y rol del actor en el momento de la acción. Este patrón se eligió deliberadamente para garantizar que la bitácora siga siendo legible y útil para auditoría aunque el usuario haya sido posteriormente eliminado, en línea con los Derechos ARCO de Cancelación que coexisten con la obligación de mantener evidencia para fines de cumplimiento.

### 3.2.2. Frontend

El frontend se construye con plantillas Blade, complementadas con Tailwind CSS para los estilos y Alpine.js para interacciones ligeras del lado del cliente. La compilación de assets se realiza con Vite. Las vistas más relevantes se ilustran a continuación en las imágenes 6 y 7.

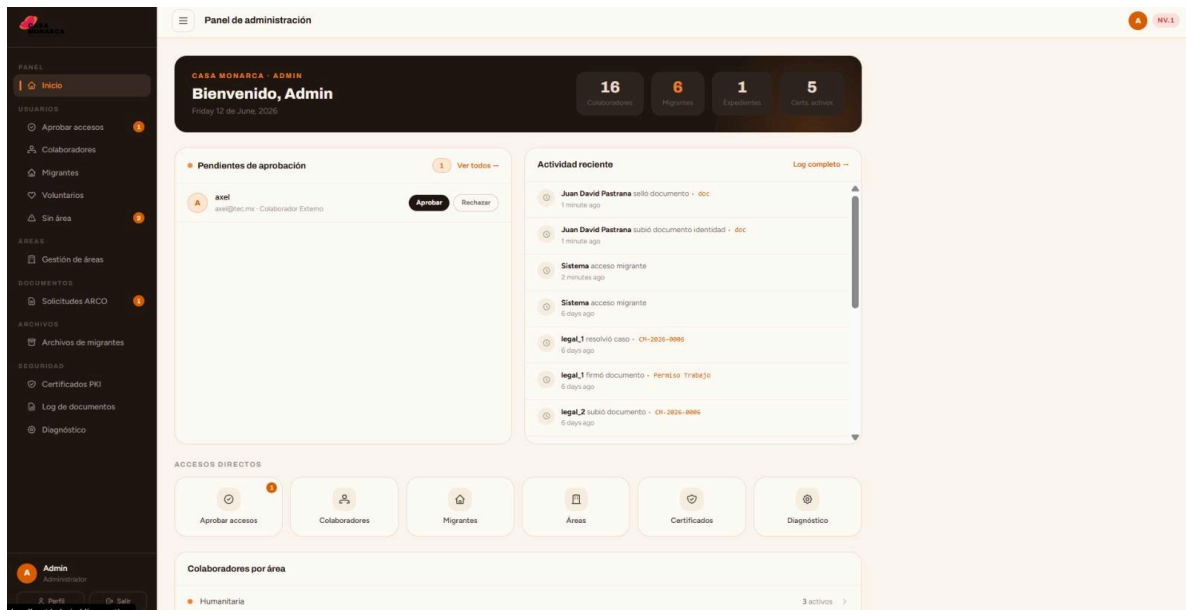


Fig. 6. Bandeja de Accesos del Administrador con solicitudes pendientes de revisión (elaboración propia).

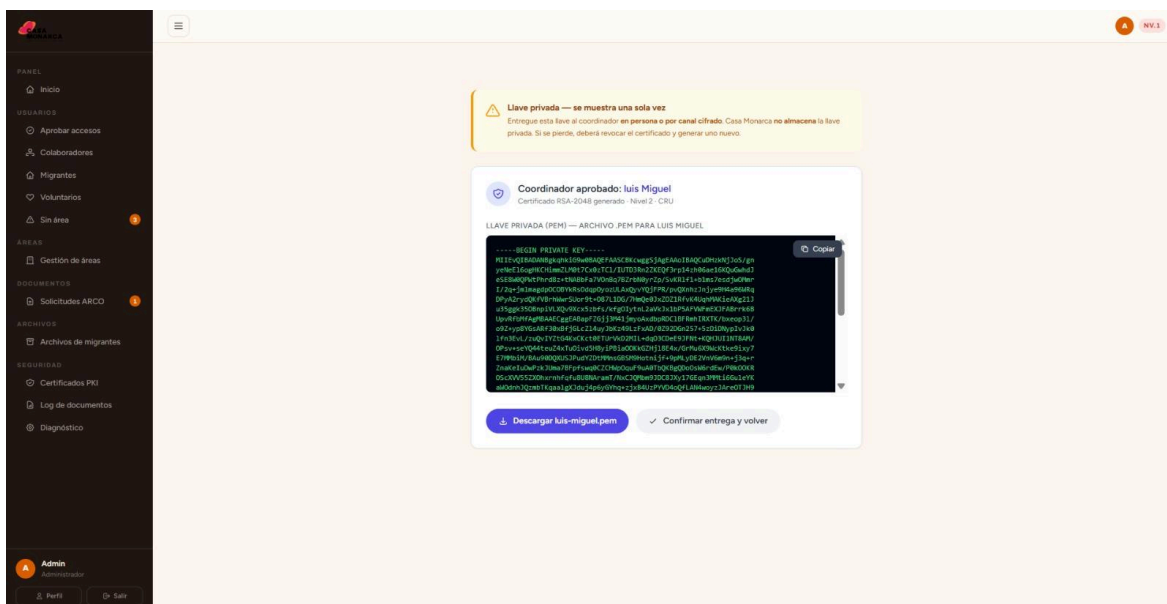


Fig. 7. Pantalla de entrega única de llave privada PEM tras aprobar a un Coordinador (elaboración propia).

### 3.2.3. Conexión de los componentes

Los componentes se conectan mediante el patrón Model-View-Controller (MVC) propio de Laravel. Las rutas web mapean cada URL a un controlador específico, los controladores delegan en servicios de negocio para operaciones complejas, los servicios consultan y modifican los modelos Eloquent, y los controladores retornan vistas Blade renderizadas con los datos resultantes. Las políticas de autorización (Gates) interceptan las peticiones para validar que el rol del usuario actual tiene permiso sobre el recurso solicitado.

### 3.3. Características adicionales en v1.1.0 (Etapa 2)

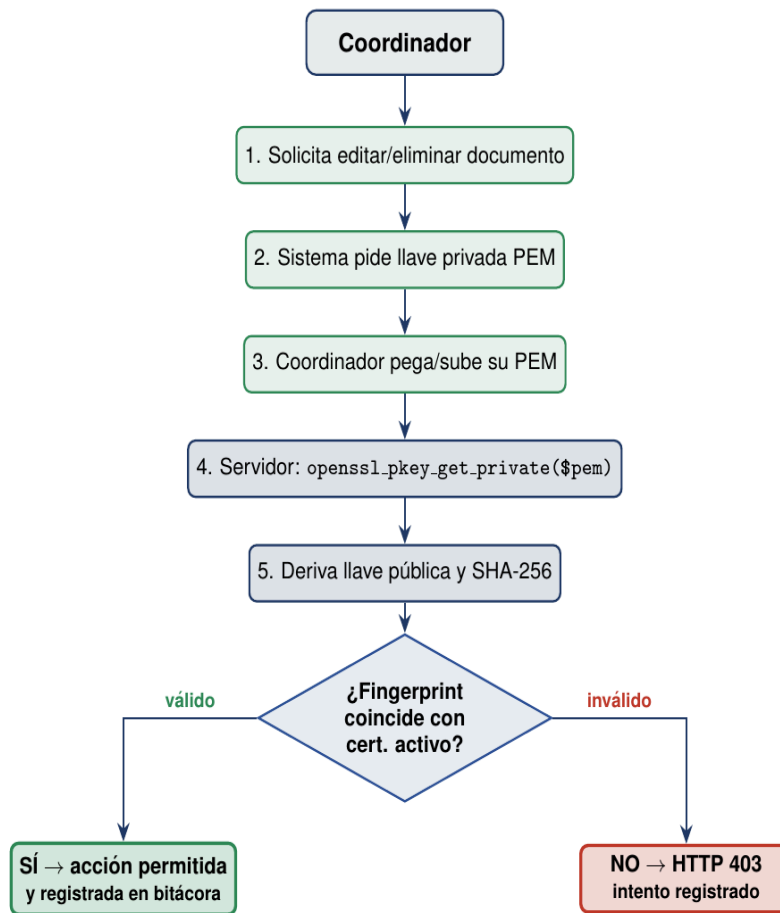
La Etapa 2 extiende el sistema con siete capacidades nuevas que cierran brechas técnicas identificadas en el piloto de la Etapa 1 y completan la cobertura operativa solicitada por Casa Monarca. A continuación se describen los componentes nuevos, organizados por dominio funcional.

#### 3.3.1. Firma digital challenge-response de documentos (FirmaController)

El módulo de firma reemplaza la verificación de posesión de llave de la Etapa 1 por una firma digital genuina sobre un challenge fresco. El flujo se implementa como un protocolo de dos peticiones HTTP coordinadas:

- **GET /firmas/{documento}/challenge:** el servidor genera un nonce aleatorio, lo asocia a la sesión del Coordinador y al ID del documento específico, y devuelve un JSON con el challenge.
- **POST /firmas/{documento}/store:** recibe el cuerpo { signature, document\_id }, recupera la llave pública del certificado activo del Coordinador, verifica la firma RSA contra el challenge previamente guardado, y, si la verificación es positiva, actualiza firmado\_at = now() y visible\_migrante = true.

En el frontend, el componente Alpine.js de firma: (1) acepta el archivo .pem por drag-and-drop, (2) lee el PEM como texto, (3) importa la llave privada con la Web Crypto API utilizando window.crypto.subtle.importKey, (4) firma el challenge con RSASSA-PKCS1-v1\_5 y digest SHA-256, (5) codifica la firma resultante en Base64 y la envía al servidor. La llave privada nunca abandona el navegador del cliente; el servidor sólo ve el challenge que él mismo generó y la firma resultante. Esta lógica puede ser visualizada a través de la figura 8.



*La comparación usa `hash_equals()` para evitar timing attacks.  
El PEM ingresado nunca se persiste en servidor.*

Fig. 8. Diagrama conceptual del protocolo challenge-response RSA-2048 — el servidor envía un nonce, el navegador firma con la llave privada local y el servidor verifica con la llave pública almacenada

Archivos clave: `app/Http/Controllers/FirmaController.php` (lógica de challenge y verificación server-side) y `resources/views/coordinador/expediente.blade.php` (componente Alpine.js con drag-and-drop y Web Crypto).

### 3.3.2. Sellado de integridad de documentos de identidad

Se añadieron dos columnas a la tabla documentos: `sello_integridad` (TEXT) y `sellado_at` (TIMESTAMP). El flujo de sellado tiene dos pasos coordinados entre la persona migrante y la aplicación:

- **store():** guarda el archivo en storage privado, calcula `hash_sha256` del contenido binario, y presenta al migrante un modal de confirmación que muestra el hash calculado para revisión.
- **sellar():** tras la confirmación explícita del migrante, calcula `sello_integridad = hash_hmac('sha256', hash_sha256, config('app.key'))` y persiste el sello junto con `sellado_at`.

La separación deliberada en dos pasos garantiza que el sello sólo se aplique cuando el migrante ha confirmado activamente que el documento subido es el correcto. La vista de auditoría /admin/archivos/{migrante} permite al Administrador realizar verificación profunda desde disco: el sistema recalcula en tiempo real el hash del archivo físico y lo contrasta con el hash registrado. Cualquier alteración posterior al sellado, ya sea por edición directa en disco, corrupción del archivo o sustitución intencional, queda detectada de inmediato. El flujo descrito puede ser visualizado a través de la figura 9.

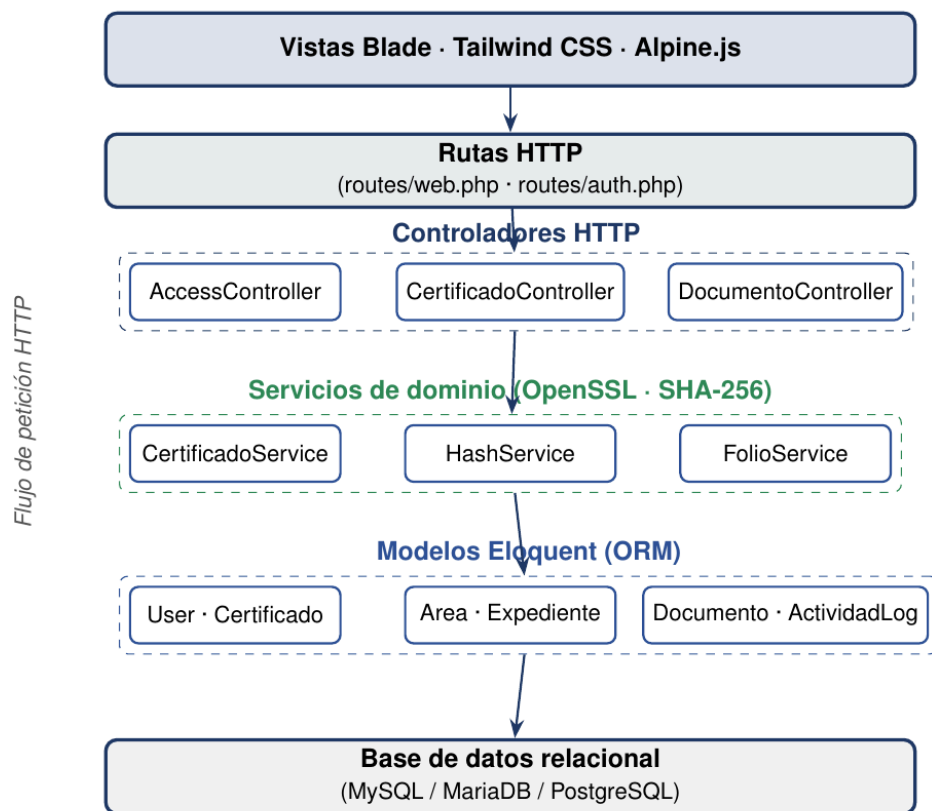


Fig. 9 Flujo de sellado HMAC-SHA256 de documentos de identidad — el migrante carga, el sistema calcula y solicita confirmación, y al confirmar se sella con la APP\_KEY como llave HMAC

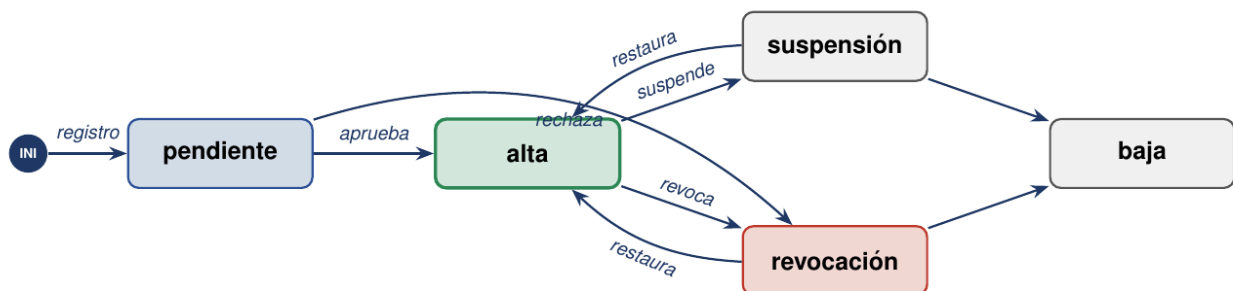
Archivos clave: app/Http/Controllers/DocumentoidentidadController.php (acciones store y sellar), app/Models/Documento.php (método selladoEsValido()), database/migrations/2026\_06\_04\_000001\_add\_sello\_integridad\_to\_documentos.php.

### 3.3.3. Flujo completo de Corrección ARCO

El modelo SolicitudRectificacion implementa la máquina de estados completa del Derecho ARCO de Rectificación. La Tabla 2 muestra los estados y los actores responsables de cada transición.

| Estado               | Quién actúa | Acción                                                                                           |
|----------------------|-------------|--------------------------------------------------------------------------------------------------|
| pendiente_correccion | Migrante    | Inicia la solicitud describiendo la corrección necesaria desde su portal.                        |
| en_proceso           | Operativo   | Toma la tarea mediante tomar() y recibe la información del documento corregido.                  |
| esperando_aprobacion | Operativo   | Sube la propuesta de corrección con subirPropuesta().                                            |
| aprobado             | Coordinador | Aprueba y firma con su .pem mediante aprobar(): se elimina el original y se activa el corregido. |

Tabla 2. Estados del flujo de Corrección ARCO (elaboración propia).



- El estado **alta** es el único que permite ingresar al sistema.
- El middleware *CheckStatus* bloquea cualquier otro estado.
- La transición a **revocación** marca todos los certificados activos del usuario como revocados.

Fig. 10. Máquina de estados del flujo de Corrección ARCO (elaboración propia).

Como podemos visualizar en la figura 10, al aprobar, el Coordinador utiliza el mismo flujo de firma digital challenge-response que para documentos de expediente. Ambas versiones del documento —la eliminada y la nueva— quedan registradas en `actividad_log` con el fingerprint del certificado que autorizó el cambio, generando un rastro auditable irrefutable. Archivos clave: `app/Http/Controllers/RectificacionController.php` y `app/Models/SolicitudRectificacion.php`.

### 3.3.4. Visibilidad en dos fases para documentos de expediente

Los documentos subidos por Operativos tienen `visible_migrante = false` por defecto. El Coordinador los ve resaltados con fondo ámbar en la vista del expediente con etiqueta "Pendiente de tu firma". Al firmar el

documento mediante el flujo challenge-response, visible\_migrante se cambia a true y el migrante puede verlo y descargarlo desde su portal. Este patrón de doble fase previene que documentos no revisados lleguen a los beneficiarios y refuerza el control criptográfico del Coordinador sobre el contenido publicado en cada expediente.

### 3.3.5. Nuevo rol: Voluntario (nivel 4)

Se añadió el nivel 4, Voluntario, a la jerarquía de roles. El rol resuelve un vacío identificado en el piloto de Etapa 1: personas que apoyan a Casa Monarca de forma puntual necesitan poder crear registros operativos pero no deben tener acceso amplio de lectura ni capacidad de modificación. La estructura completa de niveles se muestra en la Tabla 3.

| Nivel | Rol           | Permisos CRUD                                                               |
|-------|---------------|-----------------------------------------------------------------------------|
| 1     | Administrador | CRUD completo + gestión de identidades + archivos de auditoría              |
| 2     | Coordinador   | CRU en su área + firma digital de documentos                                |
| 3     | Operativo     | CR + subida de documentos y expedientes                                     |
| 4     | Voluntario    | C — sólo creación de casos                                                  |
| 5     | Migrante      | Acceso a portal propio, subida de documentos de identidad, solicitudes ARCO |

*Tabla 3. Niveles de acceso y permisos del sistema v1.1.0 (elaboración propia).*

### 3.3.6. Vista de auditoría de archivos (/admin/archivos)

Para que el Administrador pueda supervisar el estado de integridad de los documentos de identidad de los migrantes, se incorporaron tres rutas nuevas:

- **GET /admin/archivos:** galería de todos los migrantes con conteo de documentos y estado general de sellado.
- **GET /admin/archivos/{migrante}:** listado completo de documentos de identidad del migrante seleccionado, con badges de estado (sellado / no sellado) y enlace a la verificación profunda.
- **GET /admin/archivos/doc/{doc}/verificar:** endpoint AJAX que retorna JSON con el resultado de la verificación de integridad recalculando el hash del archivo desde disco.

Las descargas de archivos por parte del staff quedan registradas en actividad\_log con un prefijo del hash del documento descargado, generando trazabilidad adicional para auditoría. Archivo clave: app/Http/Controllers/ArchivosMigrantesController.php.

### 3.3.7. Creación directa de casos por Coordinador

El Coordinador puede crear casos directamente para cualquier migrante mediante la ruta GET/POST /coordinador/nuevo-caso, sin necesidad de que el migrante inicie previamente la solicitud. Esta ruta es especialmente útil para flujos de emergencia y trabajo de campo, donde la persona migrante llega al albergue sin capacidad de iniciar formalmente su solicitud (sin acceso a dispositivos, sin conocimiento del sistema, o en estado de vulnerabilidad). El caso creado se asocia inmediatamente al migrante y queda listo para asignación operativa.

### 3.3.8. Cambios de infraestructura

**Base de datos de desarrollo: SQLite.** Para simplificar el onboarding de nuevos desarrolladores, la base de datos de desarrollo cambió de MySQL 8 a SQLite (resources/database.sqlite). La base de datos de producción sigue siendo MySQL 8 accesible mediante orquestación Docker. Las migraciones de Laravel son agnósticas al motor por lo que no se requirieron cambios en los modelos.

**Vite 8.** La dependencia de Vite se actualizó de 7.x a 8.x. No hubo cambios en la configuración de vite.config.js ni en los assets compilados.

**Pest 4.4.** El framework de pruebas se actualizó a la versión menor 4.4, incorporando mejoras de rendimiento y soporte de aserciones más expresivas para los casos nuevos.

### 3.3.9. Stack tecnológico — cambios entre v1.0.0 y v1.1.0

El stack tecnológico será descrito en la tabla 4.

| Tecnología   | v1.0.0  | v1.1.0 | Cambio              |
|--------------|---------|--------|---------------------|
| Laravel      | 13.x    | 13.x   | Sin cambio          |
| PHP          | 8.3     | 8.3    | Sin cambio          |
| Tailwind CSS | 3.1+    | 3.1+   | Sin cambio          |
| Alpine.js    | 3.4+    | 3.4+   | Sin cambio          |
| Vite         | 7.0+    | 8.x    | Actualización mayor |
| Pest PHP     | 4.1     | 4.4    | Actualización menor |
| DB (dev)     | MySQL 8 | SQLite | Simplificación dev  |

| Tecnología    | v1.0.0                       | v1.1.0                      | Cambio                  |
|---------------|------------------------------|-----------------------------|-------------------------|
| DB (prod)     | MySQL 8 / VPS                | MySQL 8 / Docker            | Docker como orquestador |
| Firma digital | Verificación PEM server-side | Challenge-response JS + RSA | Nueva capacidad         |
| Roles         | 5 niveles                    | 6 niveles (+ Voluntario)    | Nivel 4 añadido         |

Tabla 4. Stack tecnológico — cambios entre v1.0.0 y v1.1.0 (elaboración propia).

### 3.3.10. Endpoints nuevos en v1.1.0

Los endpoints serán descritos en la tabla 5.

| Método | Ruta                                | Controlador@acción                    | Propósito                      |
|--------|-------------------------------------|---------------------------------------|--------------------------------|
| GET    | /firmas/{documento}/challenge       | FirmaController@challenge             | Generar nonce para firma       |
| POST   | /firmas/{documento}/store           | FirmaController@store                 | Verificar firma RSA            |
| POST   | /documentos-identidad               | DocumentIdentidadController@store     | Cargar documento de identidad  |
| POST   | /documentos-identidad/{doc}/sellar  | DocumentIdentidadController@sellar    | Aplicar HMAC-SHA256            |
| GET    | /admin/archivos                     | ArchivosMigrantesController@index     | Galería de migrantes           |
| GET    | /admin/archivos/{migrante}          | ArchivosMigrantesController@show      | Detalle por migrante           |
| GET    | /admin/archivos/doc/{doc}/verificar | ArchivosMigrantesController@verificar | Verif. integridad disco (JSON) |
| POST   | /rectificaciones                    | RectificacionController@store         | Iniciar solicitud ARCO         |

| <b>Método</b> | <b>Ruta</b>                    | <b>Controlador@acción</b>                  | <b>Propósito</b>            |
|---------------|--------------------------------|--------------------------------------------|-----------------------------|
| POST          | /rectificaciones/{r}/tomar     | RectificacionController@tomar              | Operativo toma tarea        |
| POST          | /rectificaciones/{r}/propuesta | RectificacionController@subir<br>Propuesta | Subir versión corregida     |
| POST          | /rectificaciones/{r}/aprobar   | RectificacionController@aprobar            | Coordinador firma y aprueba |
| GET           | /coordinador/nuevo-caso        | CoordinadorController@nuevo<br>CasoForm    | Form. nuevo caso            |
| POST          | /coordinador/nuevo-caso        | CoordinadorController@nuevo<br>Caso        | Crear caso directamente     |

*Tabla 5. Endpoints REST nuevos en v1.1.0 (elaboración propia).*

## 4. Resultados

### 4.1. Resultados de backend

Al cierre de la versión 1.1.0 se obtuvieron los siguientes componentes funcionales de backend:

- Servicio de emisión de certificados RSA-2048 con cálculo automático de fingerprint SHA-256 (v1.0.0).
- Middleware CheckStatus para bloqueo de cuentas no activas (v1.0.0). La pantalla resultante se ilustra en la Fig. 8.
- Servicio de verificación PEM con comparación de fingerprints en tiempo constante (v1.0.0).
- Generador de folios únicos para expedientes en formato CM-AAAA-XXXX (v1.0.0).
- Calculador de hash SHA-256 para integridad documental aplicado en cada carga de archivo (v1.0.0).
- Bitácora inmutable que preserva snapshot textual del actor (v1.0.0).
- FirmaController con protocolo challenge-response RSA-2048 completamente validado (v1.1.0).
- DocumentoidentidadController con flujo store() + sellar() y HMAC-SHA256 (v1.1.0).
- RectificacionController con máquina de estados completa del flujo ARCO (v1.1.0).

- ArchivosMigrantesController con verificación de integridad desde disco (v1.1.0).
- Migración 2026\_06\_04\_000001 que añade las columnas sello\_integridad y sellado\_at a la tabla documentos (v1.1.0).
- Nuevo rol Voluntario (nivel 4) con permisos exclusivos de creación (v1.1.0).
- Logging de descargas de staff en actividad\_log con prefijo de hash del documento descargado (v1.1.0).

Gracias al trabajo realizado en el backend, es posible que se tenga una completa seguridad sobre el sistema, y que puedan funcionar los bloqueos cuando se introducen correctamente las contraseñas y/o las llaves PEM. Esto podemos visualizarlo en la figura 11.

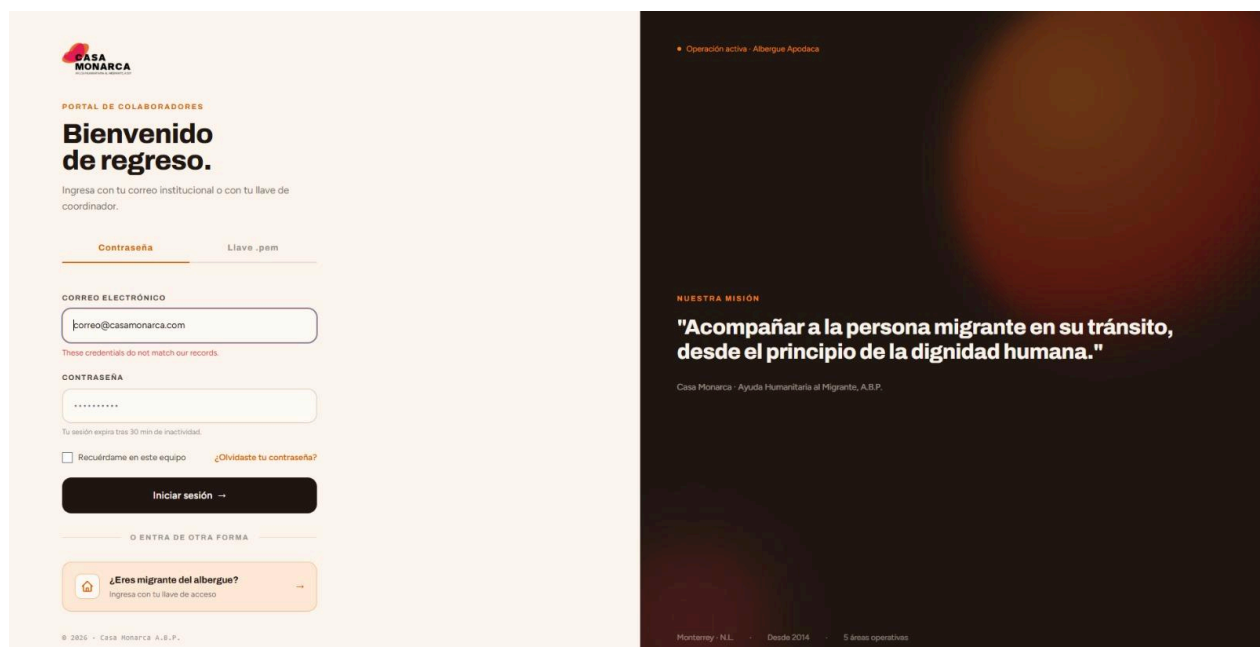
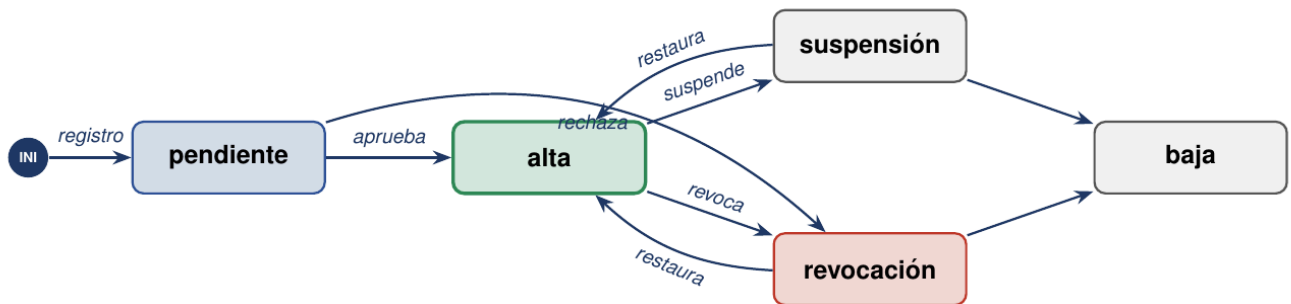


Fig. 11. Bloqueo de acceso por estado distinto de "alta" — middleware CheckStatus en acción (elaboración propia).

El sistema soporta los cinco niveles de personal interno más el rol de migrante y las seis áreas operativas, con políticas de autorización por Gate aplicadas consistentemente en todos los controladores que manejan recursos sensibles. El ciclo de vida de una identidad sigue la máquina de estados descrita en la Fig. 12.



- El estado **alta** es el único que permite ingresar al sistema.
- El middleware *CheckStatus* bloquea cualquier otro estado.
- La transición a **revocación** marca todos los certificados activos del usuario como revocados.

Fig. 12. Ciclo de vida de una identidad digital (elaboración propia).

## 4.2. Resultados de frontend

Las vistas relevantes del sistema al cierre de v1.1.0 incluyen: la Bandeja de Accesos del Administrador (Fig. 6) con filtros por estado, área y rol; la pantalla de entrega única de PEM (Fig. 7) con avisos visuales de alta visibilidad; el panel del Coordinador (Fig. 13) que muestra de forma inmediata el nivel de acceso y los permisos CRU correspondientes junto con métricas operativas del área; los formularios diferenciados de registro (Figs. 14 y 15); el componente Alpine.js de firma con drag-and-drop de .pem, firma con Web Crypto API y feedback visual de éxito/error; el modal de confirmación de sellado de documentos de identidad con UI destacada; la vista de auditoría /admin/archivos con badges de integridad y botón de verificación; la vista de expediente con documentos resaltados en ámbar pendientes de firma y flujo de firma inline; y las vistas dedicadas para Operativo y Coordinador en el flujo de rectificaciones.

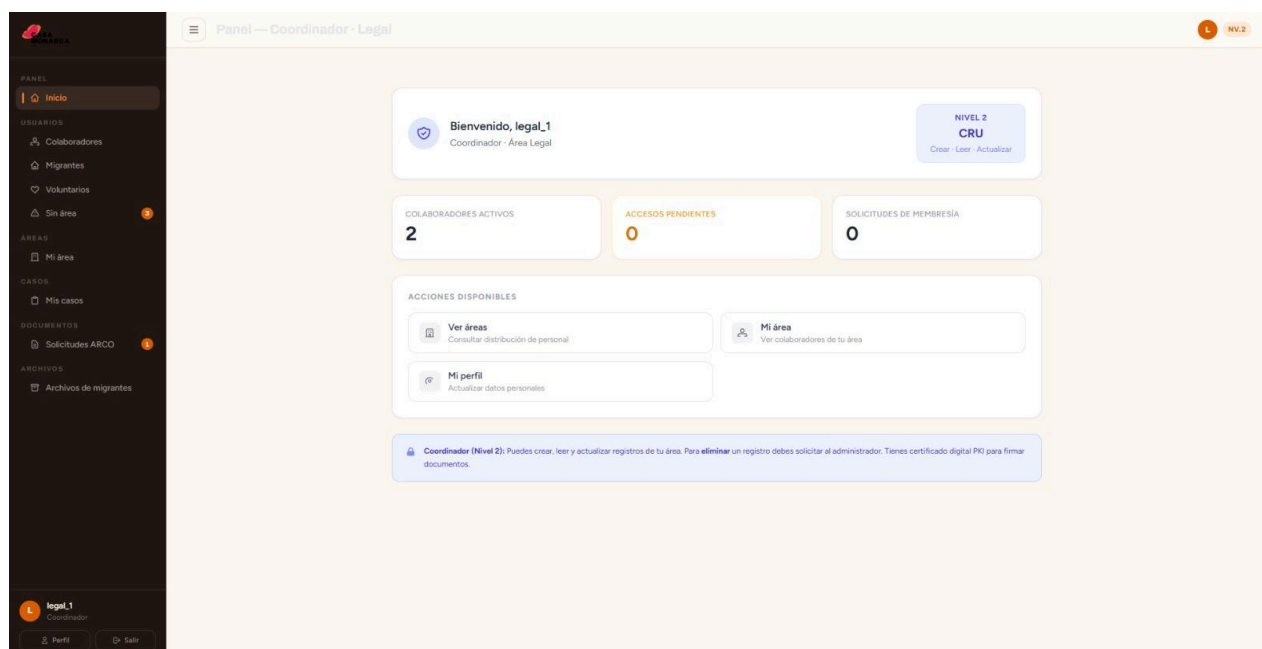
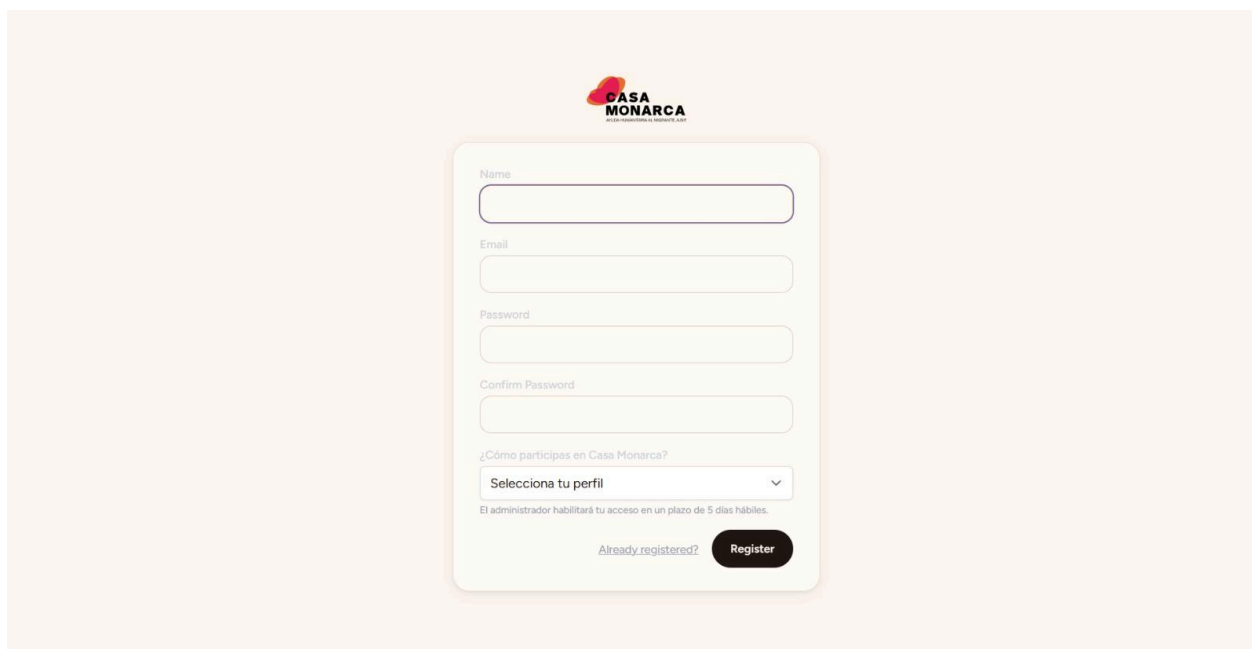
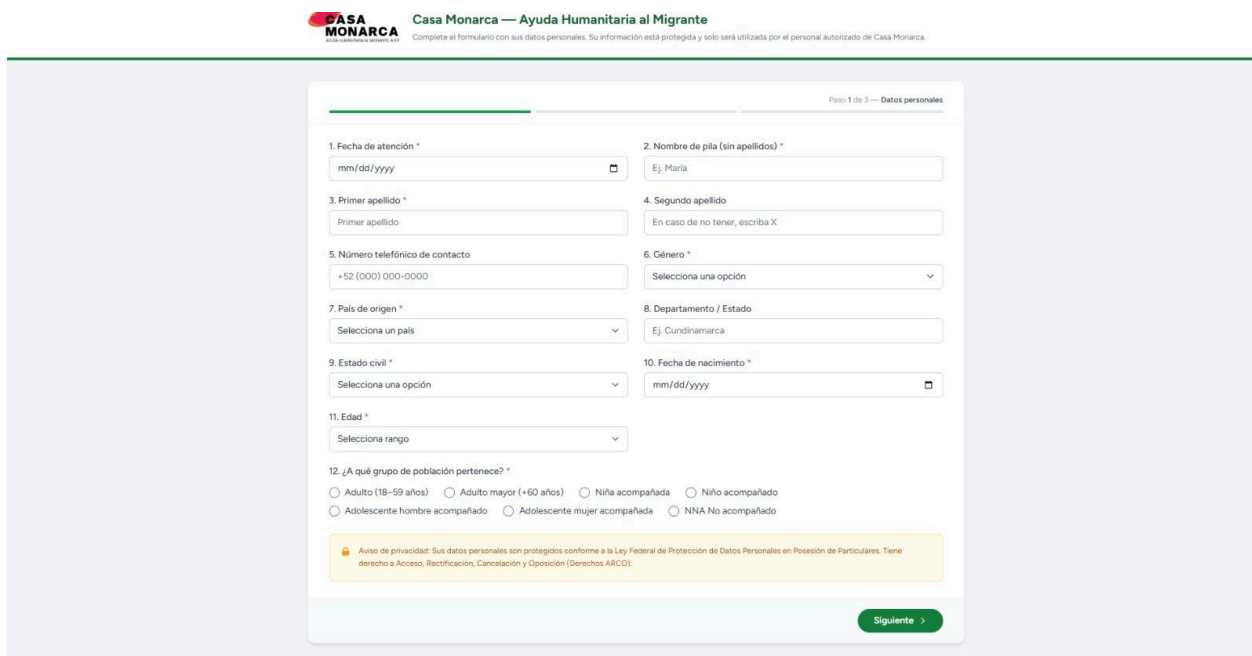


Fig. 13. Panel del Coordinador con métricas operativas y permisos visibles (elaboración propia).



Panel de registro de Casa Monarca. El formulario incluye campos para Name, Email, Password, Confirm Password, y una selección de perfil. Un mensaje indica que el administrador habilitará el acceso en un plazo de 5 días hábiles. Hay un botón 'Register' y un enlace 'Already registered?'.

Fig. 14. Formulario de registro de colaborador interno (elaboración propia).



Formulario de entrevista de ingreso al albergue para personas migrantes, paso 1 de 3. El formulario recoge datos personales como fecha de atención, nombre de pila, apellidos, primer apellido, número telefónico de contacto, género, país de origen, departamento/estado, estado civil, fecha de nacimiento, edad, y grupo de población. Incluye un aviso de privacidad y un botón 'Siguiente'.

Fig. 15. Entrevista de ingreso al albergue para personas migrantes, paso 1 de 3 (elaboración propia).

### 4.3. Resultados de la solución completa

El sistema completo, integrado y funcional en entorno de desarrollo local, ejecuta exitosamente los flujos previstos: registro de colaboradores y migrantes; aprobación con emisión de certificados; asignación de solicitudes a expedientes; carga de documentos con verificación de integridad SHA-256; firma digital

challenge-response de documentos de expediente con visibilidad diferida para el migrante; sellado HMAC-SHA256 de documentos de identidad por parte del migrante con verificación posterior desde disco por parte del Administrador; flujo completo de Corrección ARCO desde la solicitud del migrante hasta la firma del Coordinador; auditoría granular de archivos por migrante con verificación profunda; y consulta del portal migrante. Todas las acciones quedan reflejadas en la bitácora de auditoría con snapshot textual del actor. El modelo completo de la solución puede ser visualizado en la figura 16.

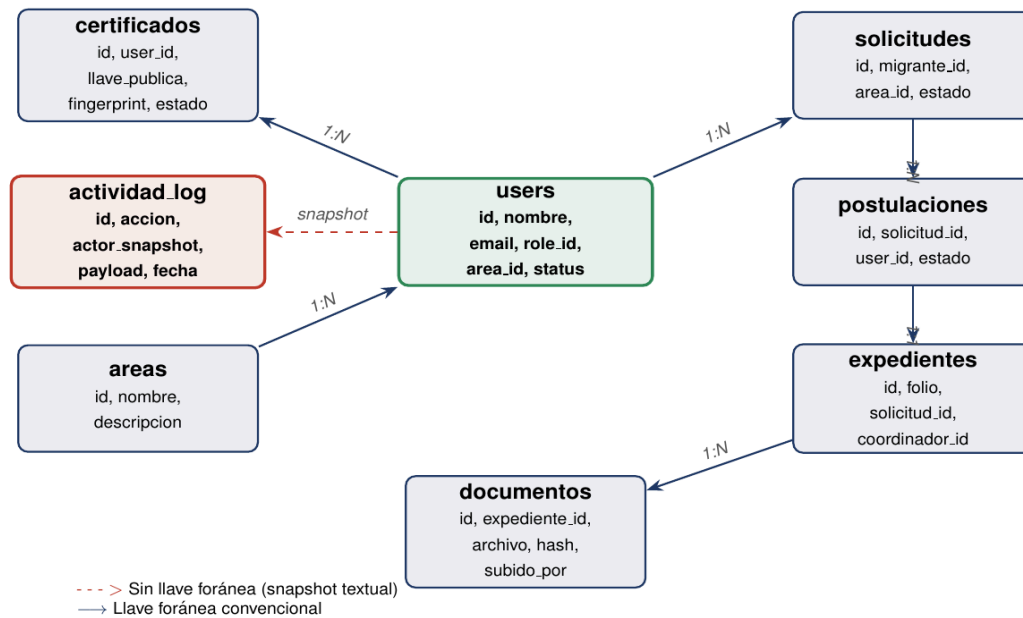


Fig. 16. Modelo entidad-relación de la base de datos (elaboración propia).

#### 4.4. Casos de uso y matriz de validación

Los veintidós casos de uso principales fueron validados manualmente y mediante el stack de pruebas Pest/PHPUnit (Pest, 2025). La Tabla 6 resume los casos validados; cada fila incluye un enlace al archivo de prueba correspondiente en el repositorio público del proyecto.

| ID    | Flujo                     | Salida esperada                       |
|-------|---------------------------|---------------------------------------|
| TC-01 | Registro de colaborador   | Usuario en estado pendiente           |
| TC-02 | Aprobación de colaborador | Usuario en estado alta                |
| TC-03 | Aprobación de coordinador | Certificado RSA-2048 + PEM única vez  |
| TC-04 | Aprobación de migrante    | Contraseña aleatoria de 10 caracteres |

| ID    | Flujo                                                          | Salida esperada                                               |
|-------|----------------------------------------------------------------|---------------------------------------------------------------|
| TC-05 | Revocación de identidad                                        | Estado revocación y certs revocados                           |
| TC-06 | Restauración de identidad                                      | Estado de regreso a alta                                      |
| TC-07 | Solicitud de migrante                                          | Solicitud pendiente registrada                                |
| TC-08 | Postulación a solicitud                                        | Postulación creada                                            |
| TC-09 | Apertura de expediente                                         | Folio CM-AAAA-XXXX generado                                   |
| TC-10 | Subida de documento                                            | Documento con hash SHA-256                                    |
| TC-11 | Edición con PEM válido                                         | Acción ejecutada con log                                      |
| TC-12 | Edición con PEM inválido                                       | HTTP 403 y log de intento                                     |
| TC-13 | Eliminación con PEM válido                                     | Documento eliminado con log                                   |
| TC-14 | Acceso sin permisos de área                                    | Acceso denegado                                               |
| TC-15 | Middleware CheckStatus                                         | Acceso bloqueado y redirección                                |
| TC-16 | Firma digital de doc. con .pem válido                          | firmado_at y visible_migrante seteados                        |
| TC-17 | Rechazo de firma con .pem de otro usuario                      | HTTP 403, log de intento                                      |
| TC-18 | Sellado de doc. de identidad por migrante                      | sello_integridad y sellado_at persistidos                     |
| TC-19 | Verificación de integridad desde /admin/archivos               | JSON con resultado consistente                                |
| TC-20 | Flujo ARCO completo: solicitud → toma → propuesta → aprobación | Documento corregido visible, original eliminado, log completo |

| ID    | Flujo                                                              | Salida esperada                      |
|-------|--------------------------------------------------------------------|--------------------------------------|
| TC-21 | Documento visible_migrante = false antes de firma; true después    | Estados consistentes pre/post firma  |
| TC-22 | Creación de caso por Coordinador sin solicitud previa del migrante | Caso creado y asociado correctamente |

Tabla 6. Matriz funcional de validación de casos de uso TC-01 a TC-22 (elaboración propia).

## 4.5. Cobertura criptográfica por servicio de seguridad

La Tabla 7 sistematiza qué algoritmos cumplen cada uno de los tres servicios clásicos de seguridad de la información (confidencialidad, integridad y autenticidad) en el sistema entregado. Esta vista consolidada es especialmente útil para auditorías y para el análisis de la cobertura técnica frente al marco normativo LFPDPPP.

| Servicio         | Algoritmo / técnica                | Aplicación                                               | Versión         |
|------------------|------------------------------------|----------------------------------------------------------|-----------------|
| Confidencialidad | bcrypt                             | Hash de contraseñas en tabla users                       | v1.0.0          |
| Confidencialidad | TLS (HTTPS)                        | Cifrado de tráfico entre navegador y servidor            | v1.0.0          |
| Confidencialidad | Patrón de no-persistencia          | Llave privada PEM nunca almacenada en servidor           | v1.0.0 / v1.1.0 |
| Integridad       | SHA-256                            | Fingerprint de llaves públicas; hash de documentos       | v1.0.0          |
| Integridad       | HMAC-SHA256                        | Sello sobre documentos de identidad (APP_KEY como llave) | v1.1.0          |
| Integridad       | Bitácora inmutable                 | actividad_log con snapshot textual del actor             | v1.0.0          |
| Autenticidad     | RSA-2048 + SHA-256 + hash_equals() | Verificación de posesión de llave privada (server-side)  | v1.0.0          |

| Servicio                  | Algoritmo / técnica                                    | Aplicación                                                | Versión |
|---------------------------|--------------------------------------------------------|-----------------------------------------------------------|---------|
| Autenticidad / No repudio | RSASSA-PKCS1-v1_5<br>+ SHA-256<br>(challenge-response) | Firma digital de documentos en cliente con Web Crypto API | v1.1.0  |

*Tabla 7. Cobertura criptográfica por servicio de seguridad (elaboración propia).*

## 5. Conclusiones y trabajo futuro

### 5.1. Conclusiones

El sistema Instar, en su versión 1.1.0, se entrega como un piloto funcional sobre desarrollo local que cubre el ciclo de vida completo de identidades digitales y la gestión integral de expedientes con firma criptográfica genuina. Los principios criptográficos del bloque MA2006B se aplicaron de manera directa y verificable: RSA-2048 para los pares de llaves de los Coordinadores, SHA-256 para fingerprints e integridad documental, HMAC-SHA256 para sellos de autenticidad sobre documentos de identidad, y el protocolo challenge-response con firma RSASSA-PKCS1-v1\_5 ejecutada por la Web Crypto API en el navegador. El patrón de entrega única de PEM, inspirado en la e.firma del SAT, garantiza que la llave privada nunca permanezca almacenada en el servidor ni durante la firma.

La elevación de la verificación PEM de la Etapa 1 a una firma digital genuina en la Etapa 2 es el cambio técnico de mayor impacto. En la v1.0.0 el sistema demostraba que el usuario poseía la llave privada correspondiente al certificado, pero no firmaba el contenido del documento; un atacante con acceso al endpoint hubiera podido reproducir la verificación sin firmar realmente nada. En la v1.1.0, el servidor genera un nonce fresco asociado al documento y a la sesión, y la firma efectivamente vincula ese nonce con la posesión de la llave privada en el momento exacto de la acción. Esta vinculación temporal y a documento específico cierra la categoría de ataques por replay y eleva el sistema a un nivel de no repudio comparable al de la e.firma.

El cumplimiento de los Derechos ARCO se aborda de manera operativa: la trazabilidad inmutable da soporte al derecho de Acceso, la versión 1.1.0 completa la cobertura del derecho de Rectificación mediante un flujo dedicado con cuatro estados y aprobación criptográfica del Coordinador, los estados de baja y revocación con bitácora persistente soportan la Cancelación, y queda pendiente para etapas siguientes la formalización del procedimiento de Oposición a usos específicos. La bitácora sin llave foránea es una decisión consciente que preserva la utilidad para auditoría aún ante la eliminación de identidades.

Adicionalmente, el sistema entrega valor más allá del Sub-problema 1 estrictamente. La gestión de áreas, solicitudes, postulaciones y expedientes con folio único; el portal trilingüe específico para migrantes; la diferenciación visual entre acciones que requieren certificado PKI y acciones que sólo habilitan acceso con contraseña; la vista administrativa /admin/archivos con verificación profunda en tiempo real; y el rol de Voluntario con permisos exclusivos de creación representan diferenciadores operativos que la organización podrá utilizar de inmediato. El rol de Voluntario, en particular, resuelve un vacío identificado en el piloto: personas que apoyan puntualmente necesitan poder crear registros sin acceso amplio de lectura ni capacidad de modificación, en línea con el principio de mínimo privilegio.

## 5.2. Trabajo futuro

Las direcciones de trabajo identificadas para etapas posteriores incluyen:

- Despliegue en hosting de producción con configuración hardenizada: HTTPS estricto, cabeceras HSTS y CSP, rotación periódica de secretos y monitoreo activo.
- Autenticación multifactor (TOTP o WebAuthn) para roles de Administrador y Coordinador.
- Procedimiento formal de Oposición ARCO con flujo dedicado en el sistema.
- Adopción del estándar X.509 RFC 5280 para los certificados internos y emisión de CRL en formato estándar.
- Integración con una Autoridad Certificadora externa para validez interinstitucional.
- Capa de registro inmutable basada en blockchain que persista únicamente hashes y fingerprints, nunca datos personales, para alcanzar inmutabilidad criptográfica de la bitácora más allá del nivel operacional actual.
- Cifrado de base de datos en reposo y respaldos automáticos cifrados, alineados con los Sub-problemas 6 y 7 del reto.
- Sistema de firma de correos electrónicos para alcanzar también el Sub-problema 2 del reto.
- Endpoint público de verificación externa de documentos para cubrir el Sub-problema 4 del reto.
- Migración del cliente de firma a hardware-backed keys (Web Authentication API con autenticadores físicos) cuando la base de coordinadores justifique la complejidad operativa.

# Referencias

Alpine.js. (2025). Alpine.js Documentation. <https://alpinejs.dev>

Barker, E. (2020). Recommendation for Key Management: Part 1 — General (NIST Special Publication 800-57 Part 1 Revision 5). National Institute of Standards and Technology.  
<https://doi.org/10.6028/NIST.SP.800-57pt1r5>

Cámara de Diputados del H. Congreso de la Unión. (2025). Ley Federal de Protección de Datos Personales en Posesión de los Particulares. Diario Oficial de la Federación.  
<https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>

Cooper, D., Santesson, S., Farrell, S., Boeyen, S., Housley, R., y Polk, W. (2008). Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile (RFC 5280). Internet Engineering Task Force. <https://doi.org/10.17487/RFC5280>

Instituto Nacional de Transparencia, Acceso a la Información y Protección de Datos Personales. (2023). Guía para titulares de los datos personales. INAI.

Krawczyk, H., Bellare, M., y Canetti, R. (1997). HMAC: Keyed-Hashing for Message Authentication (RFC 2104). Internet Engineering Task Force. <https://doi.org/10.17487/RFC2104>

Laravel. (2025). Documentación oficial de Laravel 13. <https://laravel.com/docs/13.x>

Menezes, A. J., van Oorschot, P. C., y Vanstone, S. A. (1996). Handbook of Applied Cryptography. CRC Press.

Mozilla. (2025). Web Crypto API — MDN Web Docs.  
[https://developer.mozilla.org/en-US/docs/Web/API/Web\\_Crypto\\_API](https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API)

Paar, C., y Pelzl, J. (2010). Understanding Cryptography: A Textbook for Students and Practitioners. Springer.

Pest. (2025). Pest — An elegant PHP Testing Framework. <https://pestphp.com>

Ramió Aguirre, J. (2006). Libro Electrónico de Seguridad Informática y Criptografía (6.ª ed.). Universidad Politécnica de Madrid.

Rivest, R. L., Shamir, A., y Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM, 21(2), 120–126.

Shannon, C. E. (1948). A mathematical theory of communication. Bell System Technical Journal, 27(3), 379–423.

Tailwind Labs. (2025). Tailwind CSS Documentation. <https://tailwindcss.com/docs>

The PHP Group. (2025). PHP Manual: OpenSSL extension.  
<https://www.php.net/manual/en/book.openssl.php>

# Anexos

## Anexo A. Casos de uso principales

| ID    | Actor                 | Caso de uso                     | Resultado esperado                                          |
|-------|-----------------------|---------------------------------|-------------------------------------------------------------|
| UC-01 | Colaborador prospecto | Registrar cuenta                | Cuenta en estado pendiente                                  |
| UC-02 | Administrador         | Aprobar Coordinador             | Certificado emitido y PEM mostrado una vez                  |
| UC-03 | Administrador         | Revocar identidad               | Estado revocación y certificados revocados                  |
| UC-04 | Persona migrante      | Registrar solicitud de servicio | Solicitud pendiente en bandeja del área                     |
| UC-05 | Operativo             | Postularse a solicitud          | Postulación registrada                                      |
| UC-06 | Coordinador           | Abrir expediente                | Folio CM-AAAA-XXXX generado                                 |
| UC-07 | Coordinador           | Firmar documento de expediente  | firmado_at + visible_migrante = true                        |
| UC-08 | Persona migrante      | Sellar documento de identidad   | Sello HMAC-SHA256 generado y persistido                     |
| UC-09 | Persona migrante      | Solicitar Corrección ARCO       | Solicitud en estado pendiente_correccion                    |
| UC-10 | Coordinador           | Crear caso directamente         | Caso asociado al migrante sin solicitud previa              |
| UC-11 | Administrador         | Verificar integridad de archivo | JSON con coincidencia entre hash en disco y hash registrado |

Tabla 8. Casos de uso principales del sistema (elaboración propia).

## Anexo B. Pseudocódigo de los flujos criptográficos clave

### B.1. Emisión de certificado RSA-2048 al aprobar a un Coordinador

```
function aprobarCoordinador($userId) {
  DB::transaction(function() use ($userId) {
    $user = User::findOrFail($userId);
    if ($user->role_id != Rol::COORDINADOR) return;

    // 1. Generar par RSA-2048
    $config = ["digest_alg" => "sha256",
              "private_key_bits" => 2048,
              "private_key_type" => OPENSSL_KEYTYPE_RSA];
    $key = openssl_pkey_new($config);

    // 2. Exportar partes publica y privada
    openssl_pkey_export($key, $privatePem);
    $publicPem = openssl_pkey_get_details($key)["key"];

    // 3. Fingerprint SHA-256 sobre la llave publica
    $fingerprint = hash("sha256", $publicPem);

    // 4. Persistir SOLO la llave publica
    Certificado::create([
      "user_id" => $user->id,
      "llave_publica" => $publicPem,
      "fingerprint" => $fingerprint,
      "estado" => "activo",
      "emitido_en" => now(),
    ]);

    $user->update(["status" => "alta"]);
    ActividadLog::registrar("EMITIR_CERTIFICADO", $user);

    // Llave privada -> sesion flash, UNA SOLA VEZ
    session()->flash("pem_privada", $privatePem);
  });
}
```

### B.2. Verificación challenge-response (v1.1.0)

```
// Cliente - Alpine.js + Web Crypto API
async function firmarDocumento(pemString, documentoId) {
  // 1. Pedir challenge al servidor
  const r = await fetch(`/firmas/${documentoId}/challenge`);
  const { challenge } = await r.json();

  // 2. Importar llave privada desde PEM
  const pemBody = pemString.replace(/-----[^-]+-----/g, "").trim();
  const der = Uint8Array.from(atob(pemBody), c => c.charCodeAt(0));
  const key = await crypto.subtle.importKey(
    "pkcs8", der.buffer,
    { name: "RSASSA-PKCS1-v1_5", hash: "SHA-256" },
```

```

        false, ["sign"]
    );

    // 3. Firmar el challenge
    const enc = new TextEncoder().encode(challenge);
    const sigBuf = await crypto.subtle.sign(
        "RSASSA-PKCS1-v1_5", key, enc
    );
    const signature = btoa(String.fromCharCode(...new Uint8Array(sigBuf)));

    // 4. Enviar firma al servidor
    return fetch(`/firmas/${documentoId}/store`, {
        method: "POST",
        headers: { "Content-Type": "application/json", "X-CSRF-TOKEN": csrf },
        body: JSON.stringify({ signature, document_id: documentoId })
    });
}

// Servidor - FirmaController@store
public function store(Request $r, Documento $documento) {
    $challenge = session("challenge_doc_" . $documento->id);
    abort_unless($challenge, 419, "Challenge expirado");

    $cert = auth()->user()->certificadoActivo();
    $signature = base64_decode($r->input("signature"));
    $pubKey = openssl_pkey_get_public($cert->llave_publica);

    $ok = openssl_verify($challenge, $signature, $pubKey, OPENSSL_ALGO_SHA256);
    abort_unless($ok === 1, 403, "Firma invalida");

    $documento->update(["firmado_at" => now(), "visible_migrante" => true]);
    session()->forget("challenge_doc_" . $documento->id);
    ActividadLog::registrar("FIRMAR_DOCUMENTO", auth()->user(),
        ["doc_id" => $documento->id, "fingerprint" => $cert->fingerprint]);
    return back()->with("status", "Documento firmado");
}

```

### B.3. Sellado HMAC-SHA256 de documento de identidad (v1.1.0)

```

public function sellar(Request $r, Documento $documento) {
    abort_unless($documento->migrante_id === auth()->id(), 403);

    // Calcular sello = HMAC-SHA256(hash_archivo, APP_KEY)
    $sello = hash_hmac("sha256",
        $documento->hash,
        config("app.key"));

    $documento->update([
        "sello_integridad" => $sello,
        "sellado_at" => now(),
    ]);

    ActividadLog::registrar("SELLAR_DOCUMENTO_IDENTIDAD",
        auth()->user(),

```

```

        ["doc_id" => $documento->id, "sello_prefix" => substr($sello,0,16)]);

        return redirect()->route("portal.documentos")
            ->with("status", "Sello aplicado");
    }

// Verificación: re-leer archivo de disco y comparar
public function selladoEsValido(): bool {
    if (!$this->sello_integridad) return false;
    $hashActual = hash_file("sha256", Storage::path($this->archivo));
    if (!hash_equals($this->hash, $hashActual)) return false;
    $selloRecalc = hash_hmac("sha256", $hashActual, config("app.key"));
    return hash_equals($this->sello_integridad, $selloRecalc);
}

```

## Anexo C. Stack tecnológico completo v1.1.0

| Capa                | Tecnología                     | Versión      |
|---------------------|--------------------------------|--------------|
| Lenguaje principal  | PHP                            | 8.3          |
| Framework           | Laravel                        | 13.x         |
| Autenticación       | Laravel Breeze                 | 2.3          |
| ORM                 | Eloquent (incluido en Laravel) | —            |
| Vistas              | Blade                          | —            |
| CSS                 | Tailwind CSS                   | 3.1+         |
| Interacción cliente | Alpine.js                      | 3.4+         |
| Firma client-side   | Web Crypto API                 | estándar W3C |
| Compilación         | Vite                           | 8.x          |
| Criptografía server | OpenSSL nativa de PHP          | —            |
| DB desarrollo       | SQLite                         | 3.x          |
| DB producción       | MySQL                          | 8.x (Docker) |
| Pruebas             | Pest / PHPUnit                 | Pest 4.4     |

| Capa                 | Tecnología   | Versión |
|----------------------|--------------|---------|
| Formato de código    | Laravel Pint | 1.24    |
| Control de versiones | Git + GitHub | —       |

*Tabla complementaria: Stack tecnológico completo al cierre de v1.1.0 (elaboración propia).*

## Anexo D. Glosario técnico

**PEM.** Privacy-Enhanced Mail. Formato de codificación ASCII de objetos criptográficos (llaves, certificados) que envuelve datos binarios DER en Base64 con delimitadores estándar BEGIN/END.

**Fingerprint.** Resumen criptográfico (SHA-256 en este sistema) de la representación PEM de una llave pública. Sirve como identificador corto y único de un certificado.

**Challenge.** Nonce aleatorio generado por el servidor para una sola autenticación, asociado a la sesión y al recurso específico. Cierra la categoría de ataques por replay.

**HMAC.** Hash-based Message Authentication Code. Construcción que combina una función hash con una llave secreta para producir un sello que sólo puede ser generado y verificado por quien conoce la llave.

**PKI.** Public Key Infrastructure. Conjunto de roles, políticas y procedimientos para gestionar el ciclo de vida de certificados digitales.

**ARCO.** Acceso, Rectificación, Cancelación y Oposición. Los cuatro derechos del titular de datos personales reconocidos por la legislación mexicana.

## Anexo E. Link de Github

<https://github.com/A01751471/casa-monarca>